

Laboratory work 6

Functions

Objective: to learn means of describing user functions, to learn apply user functions.

Short theoretical information

6.1. General view of the function.

A function is an independent piece of program code that can be used multiple times in a program. The general appearance of the function is as follows:

```
return_value_type function_name (argument_list)
{
the body of the function
}
```

The `return_type` specifies the type of variable returned by the function. A function can return a variable of any type except an array. The `void` type is used to declare a function that does not return a value.

The `parameter_list` contains the argument types and their names, separated by commas (these are the formal parameters of the function). If the function has no arguments, `parameter_list` is empty (in which case the parentheses are empty or contain the `void` keyword). Each parameter of the function is declared separately, the type and name must be specified for each parameter. The general form of the declaration of function parameters in its header is as follows:

```
type variable_name1, type variable_name2, ..., type variable_nameN
```

A function must be declared using a function prototype before being used. The general appearance of the function prototype is as follows:

```
type function_name (type parameter_name1, type parameter_name2, ...,
parameter_name typeN);
```

If the function has no parameters, the keyword `void` must be specified in its prototype instead of the parameter list. If the function is defined before its use, a special prototype is not required. However, for large programs, it is desirable to declare using prototypes functions.

A set of actual parameters is specified at the point of the function call.

6.2. Passing parameters by value and by reference.

Parameters can be passed in two ways: by value and by reference. When passing parameters by value, a copy of the argument value is assigned to the formal parameter. In this case, parameter changes do not affect the argument. Usually, in the C language, parameter-by-value transfer is used (with the exception

of cases when an array is used as a function argument).

Example:

```
#include <stdio.h>
int cub(int a);
int main(void)
{
    int b = 3;
    printf("%d %d ", cub(b), b);
    return 0;
}
int cub(int a)
{
    a = a*a*a;
    return(a);
}
```

The result of the program:

27 3

When passing parameters by reference, a pointer to the argument is passed to the function. A pointer is a variable whose value is the address in computer memory of a specific object (usually another variable). In this case, the address of the argument is copied to the parameter, therefore, changes in the value of the parameter lead to changes in the value of the argument. The general form of the index declaration is as follows:

type *name;

The base type of a pointer is determined by the type of object to which the pointer will refer.

A function that uses parameters as pointers must be called with the addresses of the arguments.

Example:

```
#include <stdio.h>
void swap(int *c, int *k);
int main(void)
{
    int a, b;
    a = 0; b = 1;
    swap (&a, &b);
    printf("a=%d b=%d ", a, b);
    return 0;
}
void swap (int *c, int *k)
{
    int tmp;
    tmp = *c; /*save the value at address c */
```

```

*c = *k; /* put k in c */
*k = tmp; /* put c in k */
}

```

The result of the program:

a=1 b=0

6.3. Returns from functions and return values.

All functions, except those belonging to the void type, return the value specified by the expression in the return statement. The return statement is used to exit the function and go to the point where it was called.

Example:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int sqr(int a);
int main(void)
{
    int b;
    scanf("%d", &b);
    printf("%d", sqr(b));
    return 0;
}
int sqr(int a)
{
    return a*a;
}

```

The return operator is used in functions of the void type without meaning. The closing curly brace (}) also causes an exit from functions. In this case, the non-void type function returns an arbitrary value. A function that has a type other than void can be used as an operand in expressions, for example:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int cub(int a);
int main(void)
{
    int y;
    scanf("%d", &y);
    if (cub(y) < 1000)
        printf("insufficient\n");
    return 0;
}
int cub(int a)
{
    a = a*a*a;
}

```

```
return(a);  
}
```

The function can have any number of return statements,
example:

```
#define _CRT_SECURE_NO_WARNINGS  
#include <stdio.h>  
#include <locale.h>  
int func(int x);  
int main(void)  
{  
    setlocale(LC_ALL, "ukr");  
    int b;  
    scanf("%d", &b);  
    if (func(b)) printf("Число відмінне від 0\n");  
    return 0;  
}  
int func(int x)  
{  
    if (x>0) return 1;  
    else if (x<0) return -1;  
    else return 0;  
}
```

6.4. Passing arrays as function parameters.

If the function argument is a one-dimensional array, then the formal function parameter can be declared as a pointer, as a fixed-size array, and as an undefined-size array, for example:

```
void f1(int *mas)  
{  
    /* ... */  
}  
or  
void f1(int mas[100])  
{  
    /* ... */  
}  
or  
void f1(int mas[])  
{  
    /* ... */  
}
```

To pass an array as an argument to a function, you need to pass a pointer to the array. An array name without an index is a pointer to its first element.

```

Example:
int main(void)
{
    int mas[100];
    f1(mas);
    /* ... */
}

```

In the parameter of the function, which receives a two-dimensional array, the size of the right dimension, which is equal to the length of the row of the array, must be specified. It is not necessary to specify the size of the left dimension.

```

Example:
void f1(int mas[][10])
{
    /* ... */
}

```

If a two-dimensional array is used as a function argument, a pointer to the initial element of the array is passed to the function.

Example 6.1.

Write a function that outputs the elements of a one-dimensional array to the console.

Program text

```

#include <stdio.h>
#define N 10
void print_m(int *m);
int main(void)
{
    int mas[N] = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 };
    print_m(mas);
    return 0;
}
void print_m(int *m)
{
    for (int i = 0; i < N; ++i)
        printf("%d\n", m[i]);
}

```

Example 6.2.

Write a function that calculates the sum of elements array

Program text

```

#include <stdio.h>

```

```

#define N 10
int sum_m(int *m);
int main(void)
{
    int mas[N] = { 10, 20, 30, 40, 50,
60, 70, 80, 90, 100 };
    printf("s=%d\n", sum_m(mas));
    return 0;
}
int sum_m(int *m)
{
    int s = 0;
    for (int i = 0; i < N; ++i) s +=
m[i];
    return s;
}

```

Tasks of laboratory work

Task 1

A given two-dimensional array of $n \times t$ elements. Write a program that, with separate functions, performs actions according to the option:

- 1) fills the array with random numbers in the range from 1 to 50 and finds the value of the minimum array element;
- 2) fills the array with values entered from the keyboard, and finds the sum of array elements;
- 3) fills the array with values entered from the keyboard, and finds the value of the maximum array element;
- 4) fills the array with random numbers in the range from 1 to 50 and finds the product of array elements.

Task 2

Develop an algorithm and program using recursion to calculate function values. The argument of the function belongs to a specific given set of natural numbers. Task options are listed in Table 6.1.

Table 6.1

Variant	Function	Analytical expression	Conditions
1	$f(x)$	$\begin{cases} 1 \\ 5 \cdot f(x-1) \end{cases}$	$\begin{matrix} x = 1 \\ x > 1 \end{matrix}$
2	$f(x)$	$\begin{cases} 5 \\ \ln(f(x-1) - x) \end{cases}$	$\begin{matrix} x = 3 \\ x > 3 \end{matrix}$
3	$f(x)$	$\begin{cases} 6 \\ f(x-1) + 2^x \end{cases}$	$\begin{matrix} x = 1 \\ x > 1 \end{matrix}$
4	$f(x)$	$\begin{cases} 3 \\ f(x-1)/3 \end{cases}$	$\begin{matrix} x = 6 \\ x > 6 \end{matrix}$
5	$f(x)$	$\begin{cases} 6 \\ f(x-1)/2 - 3x \end{cases}$	$\begin{matrix} x = 3 \\ x > 3 \end{matrix}$

Task 3.

Write a program for determining the value of a function at a point. It provides for the implementation of three functions: the first provides input of information, the second - calculation of the required value, the third - display of results. Transfer parameters to organize without using global variables.

Task options:

$$1) d = (a^2 + \cos b^2) \cdot \sin^2 x + y / \sqrt{a}$$

$$2) y = \frac{1}{\sin a^3 \sqrt{(m+3)} / p^2} - \frac{5bc+d}{\cos x};$$

$$3) y = \frac{\sqrt{a} \sqrt{\pi} + e^{bx}}{bx^2 + az - \cos^{bx}};$$

$$4) Z = \frac{\ln(x/y) + \sqrt{ax^2 + b} + j}{e^{-x/y} \sin(x^2 + y)};$$

$$5) f = \frac{\sqrt{a} + \sin(x/y) + a\sqrt{x^2 + y^2}}{\ln(x + y - k)}.$$